Smart-RUDP

Reliable UDP with learning-based congestion control, in C

Computer Networks | Faculty: Anuradha G

Harshavardhan Mallela 24BAI0274 Rohit Arya 24BCE0819 Shloak Sinha 24BDS0378

The problem

UDP

Fast and simple, but no guarantees: packets can be lost, duplicated or reordered.

TCP

Reliable, but lives inside the OS kernel. Hard to see, hard to change, hard to learn from.

In 1986 the early Internet suffered congestion collapse: throughput fell from 32 Kbps to 40 bps. TCP congestion control was invented to fix it.

Our question: can we rebuild TCP's core ideas ourselves, on top of UDP?

Why we built it

The need

Campus users send data to a server across a routed network. TCP is fixed inside the kernel; plain UDP gives no guarantees. Real systems (QUIC, HTTP/3) solve this with reliability built in user space over UDP.

Our answer

RUDP: a readable, tunable reliable transport over UDP, plus Smart-RUDP, a controller that learns the window from RTT and loss.

Scenario: a student on VLAN 10 uploads a file to the server LAN. The ACL keeps students out of staff VLAN 20. If a core link fails, OSPF reroutes through R2: a longer path and higher RTT that the transport must adapt to.

Validated: protocol tests and Smart-RUDP vs AIMD on emulated paths; network design built and tested in Packet Tracer; traffic analysed in Wireshark.

What we built

Handshake

SYN, SYN-ACK, ACK before any data flows.

Reliability

Sequence numbers, ACKs, checksums, timeouts, retransmission.

Flow control

Go-Back-N sliding window; its size is the congestion window.

Congestion control

AIMD: slow start, avoidance, halve on loss.

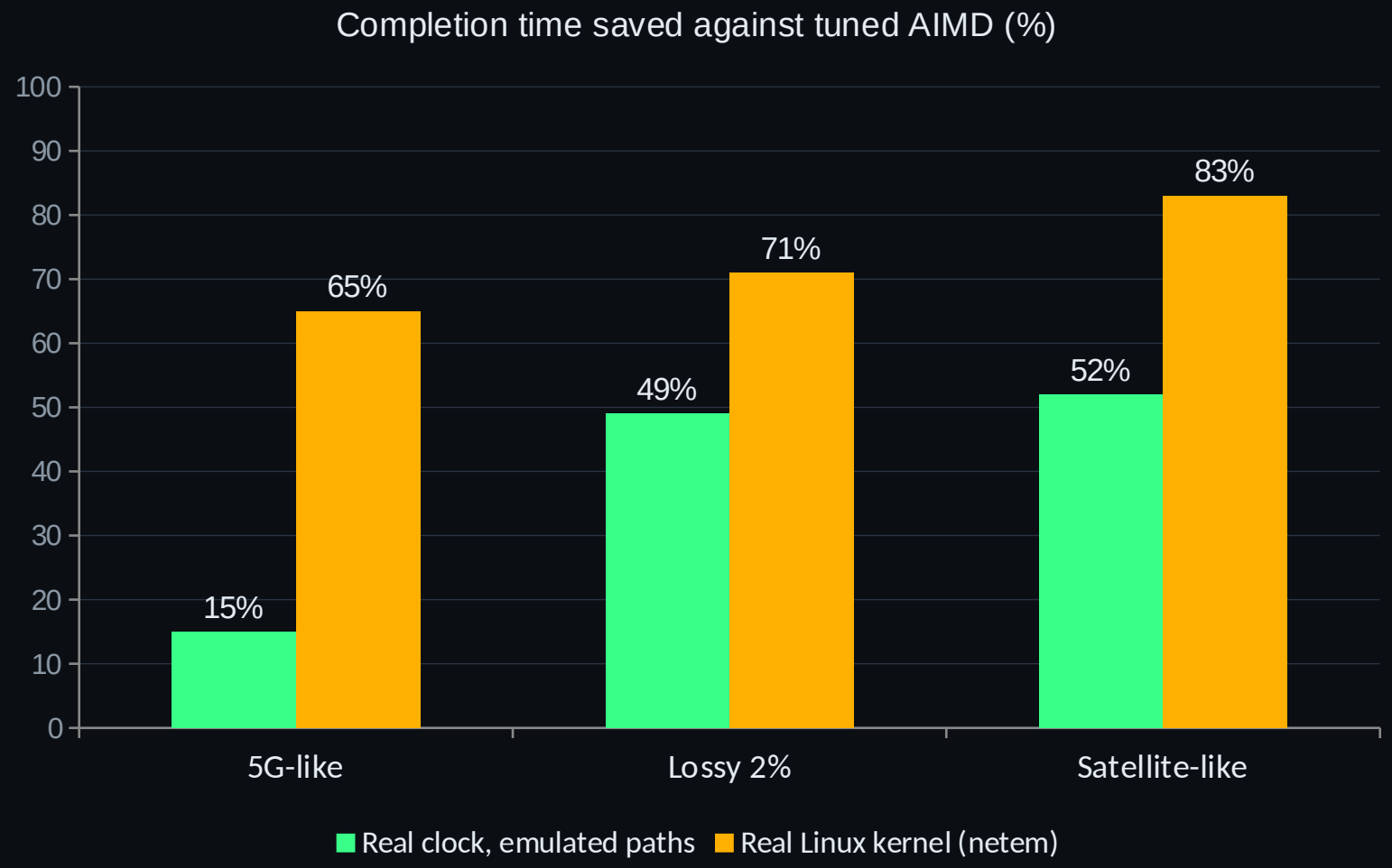
About 670 lines of C across 4 source files. Client + server, built in 8 phases, on github.com/Capsturrrr/rudp

Results at a glance

What we measured	Result
Five named paths vs tuned AIMD (real clock)	15% to 52% faster on four paths; a tie on the hostile 10%-loss path
100 random held-out paths	0.53x the time of tuned AIMD; faster on 89; never more than 20% slower
Real Linux kernel (netem)	60% to 83% faster than tuned AIMD (the reordering path is not significant)
Paths that change mid-transfer	Fastest in 4 of 4 cases
Two flows with unequal RTT	Jain fairness 0.84, against 0.64 for AIMD
Next to a kernel TCP flow	Takes 24% to 34% of the link; does not starve TCP
Between two real laptops	Works over the internet; every completed transfer delivered in full
The whole system	OSPF network, Wireshark dissector, chat over the internet, RUDP Drop app

Caveat, covered later: kernel TCP is still 1.5x to 1.7x faster than our agent, and the first versions had problems we found and fixed.

Up to 83% less time than tuned AIMD



- Same trained network on every path, no retuning
- Real-clock runs: 10 paired runs per path, 95% intervals
- Kernel runs: 3 to 5 per cell, significant on all three paths shown
- Also faster on the fast path (52% real clock), and 0.53x the time on 100 random paths

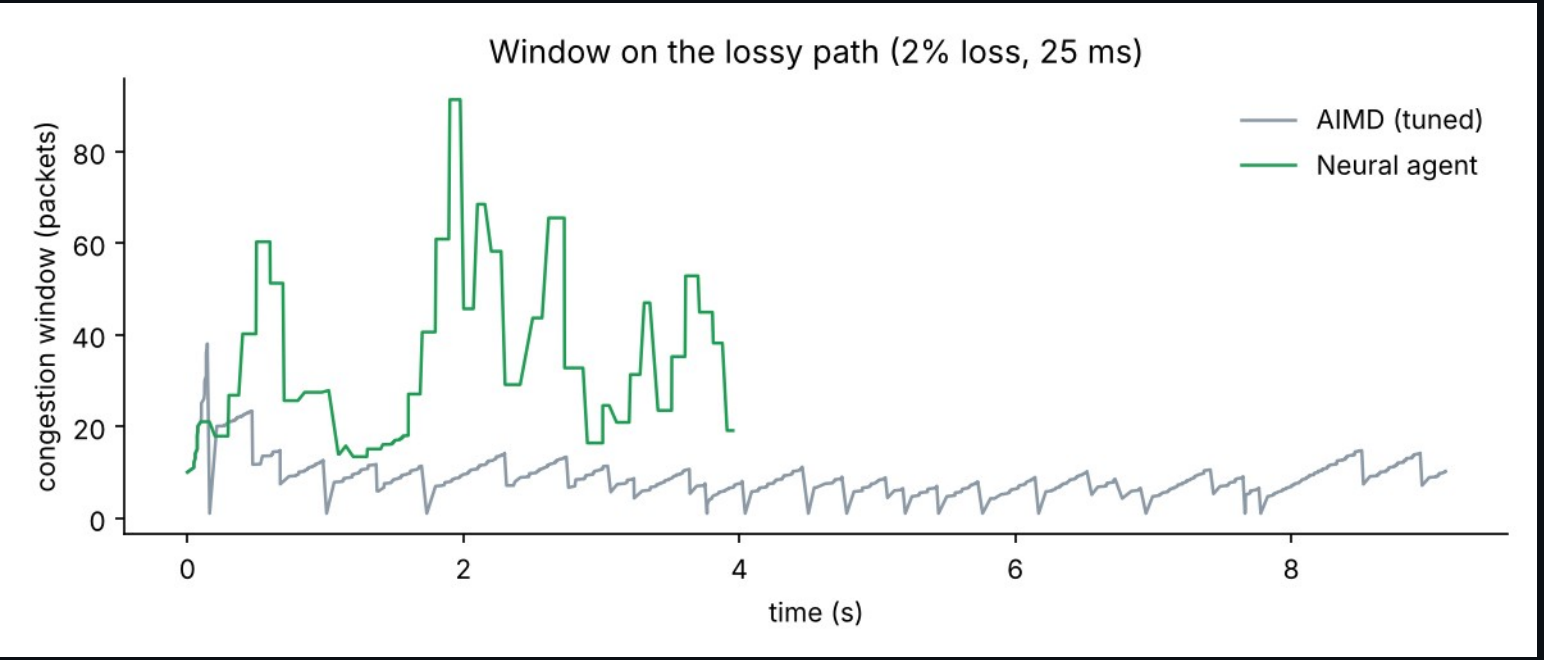
Real clock: 15% to 52% faster than tuned AIMD

Path (seconds)	AIMD cap 32	AIMD tuned	CUBIC tuned	Neural
5G-like	1.41	1.09	1.19	0.92
Lossy 2%	7.98	7.05	13.92	3.62
Satellite-like	17.93	11.36	8.26	5.46
Fast (3,000 pkts)	14.55	8.20	6.52	3.92
Hostile 10% loss	7.54	8.34	12.32	8.93

- Neural agent fastest on 4 of 5 paths; significantly faster than tuned AIMD on all four: -15%, -49%, -52%, -52%
- Faster than tuned CUBIC on lossy (-74%) and satellite (-34%); equal on 5G-like and fast; CUBIC is 2x slower than AIMD at 2% loss
- Hostile 10% loss: no significant difference between any two policies
- Retransmissions now low (3, 6, 4 packets): the old 262 were spurious timeouts, fixed with a timer floor that AIMD also benefits from

Real clock, real server process, 10 paired runs per path (hostile: 5), 95% intervals. eval_final.sh. Every policy uses the same timer, window cap rules and Selective Repeat.

Seeing the difference: the window



- Same lossy path, same seed, same transport
- AIMD (grey) saws between 5 and 15 packets and needs 9 s
- The agent (green) rides much higher, backs off on loss, and finishes in 4 s
- Regenerate: make figures

Real Linux kernel: 65% to 83% faster

Path (500 packets)	TCP	AIMD tuned	Neural	vs AIMD
Lossy, 2% loss	0.88 s	4.75 s	1.35 s	-71%
5G-like	0.28 s	1.23 s	0.43 s	-65%
Satellite, 150 ms (3 runs)	1.91 s	16.7 s	2.92 s	-83%
Reordering	0.94 s	4.06 s	1.64 s	-60% (n.s.)

- Linux kernel tc/netem on loopback, 5 runs per cell, 512-byte payloads for TCP, UDP and RUDP
- The neural agent beats tuned AIMD on a real kernel path: 65% to 83% less time, significant on three paths
- Sharing a link with a kernel TCP flow: the agent takes 24% to 34% and TCP keeps the rest, so it does not starve TCP (AIMD gets only 5% to 17%)
- The same trained network as in every other test, with no retuning for the kernel

Changing paths: fastest in 4 of 4 cases

Mean time (s)	AIMD tuned	CUBIC	BBR-style	Neural
Bandwidth drop	13.6	17.7	13.4	12.7
Bandwidth rise	9.2	15.1	5.0	4.3
Delay jump	33.3	27.5	25.9	19.8
Hand-over pattern	12.8	21.4	10.8	8.2
2% lossy path (3,000 pkts)	19.8	53.3	8.2	9.8
Satellite-like path	32.8	26.6	16.8	11.2

- Paths that change mid-transfer (bandwidth drop or rise, delay jump): the neural agent is fastest in all 4 cases
- Against a BBR-style baseline: the neural agent wins on satellite-like (11.2 s vs 16.8 s) and fast (3.7 s vs 6.2 s) paths and is level on 5G-like
- The BBR-style baseline itself beats tuned AIMD on 4 of 5 paths, so the comparison is against a strong opponent
- Virtual time, 20 paired seeds; a simplified BBR, not the Linux one

Fairer than AIMD, and does not starve TCP

Two identical flows	Extra RTT	Share short-RTT	Jain
AIMD / AIMD	150 ms	88%	0.64
Neural / neural	150 ms	71%	0.84

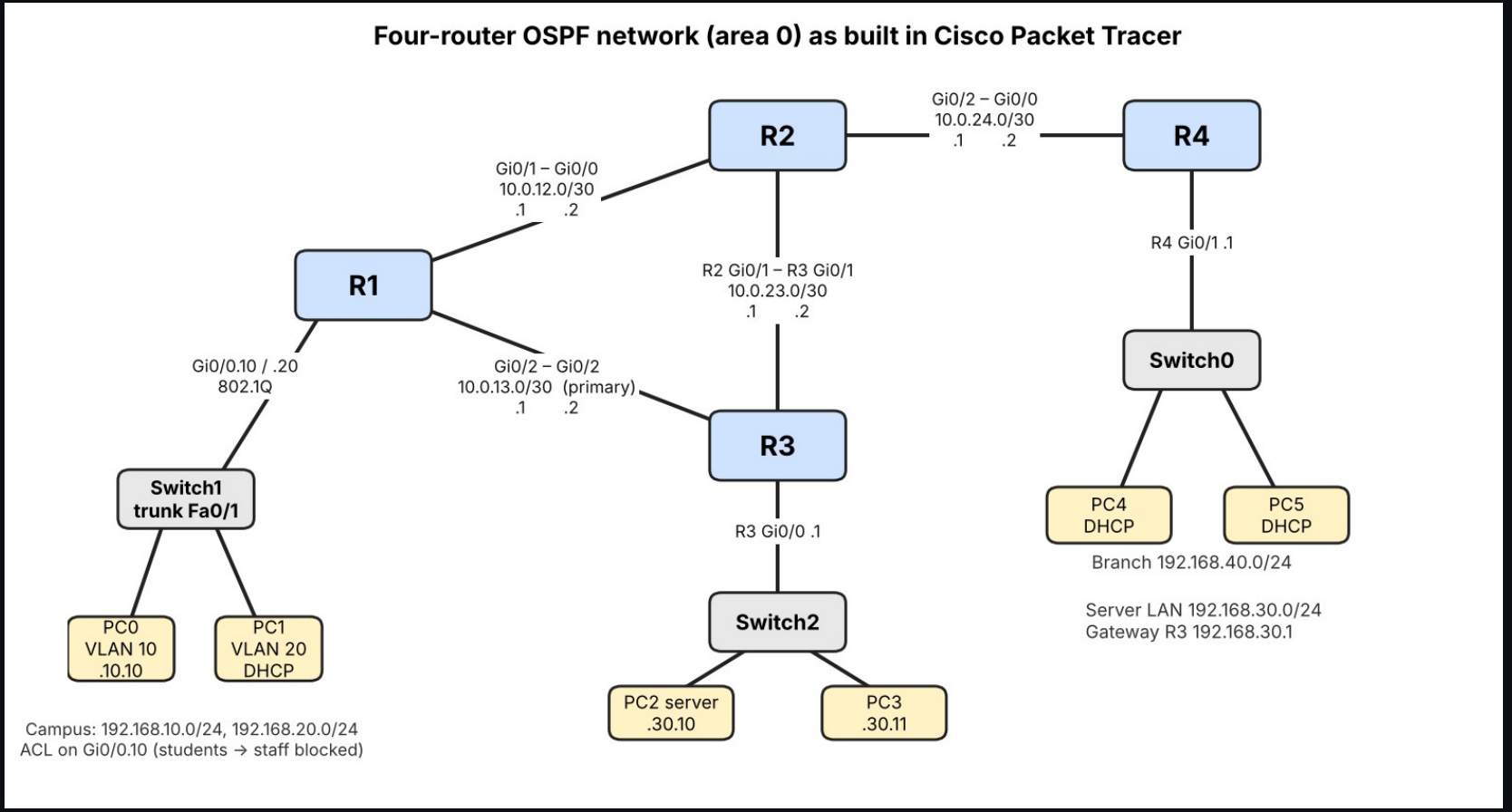
- Between two flows with unequal RTTs the neural agent is fairer than AIMD (Jain 0.84 against 0.64; 6 pairs per cell)
- Next to a kernel TCP flow it takes 24% to 34% of the link, so it does not starve TCP
- Robust in practice: noise of 0.2 on its inputs costs at most 12%
- A depth-5 decision tree reproduces its transfer time on 4 of 5 paths, so its behaviour can be summarised by a small rule set

Works between two real machines

Time (s)	TCP	AIMD	CUBIC	BBR-style	Neural
Rep 1 (clean)	2.5	14.2	12.4	6.4	4.7
Rep 2 (clean)	2.0	12.0	-	-	-
Rep 3	39.3	-	240	53	71
Rep 4	11.4	87	218	69	111
Rep 5	50.9	143	167	44	32

- Two laptops, different networks, Tailscale relay (about 79 ms RTT); 2 MB per transfer, 5 repetitions
- RUDP works between real machines: every completed run delivered 4,000 of 4,000 packets
- The path drifted (TCP alone 2 s to 51 s) and 4 of 20 RUDP runs timed out, so this is a demonstration, not a ranking
- Clean reps: kernel TCP 2x to 6x faster; rep 1 the neural agent beat AIMD (4.7 s vs 14.2 s)
- Next: repeat on a direct local network with more reps

Network design: OSPF topology in Packet Tracer



- 4 routers, OSPF area 0 (R1-R2-R3 triangle, R4 branch)
- 2 VLANs, router-on-a-stick, DHCP on R1 and R4
- Access list: student to staff blocked (verified)
- Routes: server LAN metric 2 via R3, branch metric 3 via R2
- Failover: shut R1 Gi0/2, 70/71 pings, reroute via R2 (metric 3)

Built and tested in Packet Tracer: OSPF adjacencies, routes, ACL and failover all verified.

Packet Tracer results: routes, failover, ACL

```
Neighbor ID     Pri   State           Dead Time   Address         Interface
3.3.3.3           1   FULL/DR         00:00:36    10.0.13.2       GigabitEthernet0/2
R1#show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0     unassigned      YES unset  up                    up
GigabitEthernet0/0.10  192.168.10.1    YES manual up                    up
GigabitEthernet0/0.20  192.168.20.1    YES manual up                    up
GigabitEthernet0/1     10.0.12.1       YES manual up                    up
GigabitEthernet0/2     10.0.13.1       YES manual up                    up
Vlan1                  unassigned      YES unset  administratively down down
R1#
01:06:43: %OSPF-5-ADJCHG: Process 1, Nbr 2.2.2.2 on GigabitEthernet0/1 from LOADING to FULL,
Loading Done

01:09:03: %OSPF-5-ADJCHG: Process 1, Nbr 2.2.2.2 on GigabitEthernet0/1 from FULL to DOWN,
Neighbor Down: Dead timer expired

01:09:03: %OSPF-5-ADJCHG: Process 1, Nbr 2.2.2.2 on GigabitEthernet0/1 from FULL to DOWN,
Neighbor Down: Interface down or detached

01:09:03: %OSPF-5-ADJCHG: Process 1, Nbr 3.3.3.3 on GigabitEthernet0/2 from FULL to DOWN,
Neighbor Down: Dead timer expired

01:09:03: %OSPF-5-ADJCHG: Process 1, Nbr 3.3.3.3 on GigabitEthernet0/2 from FULL to DOWN,
Neighbor Down: Interface down or detached

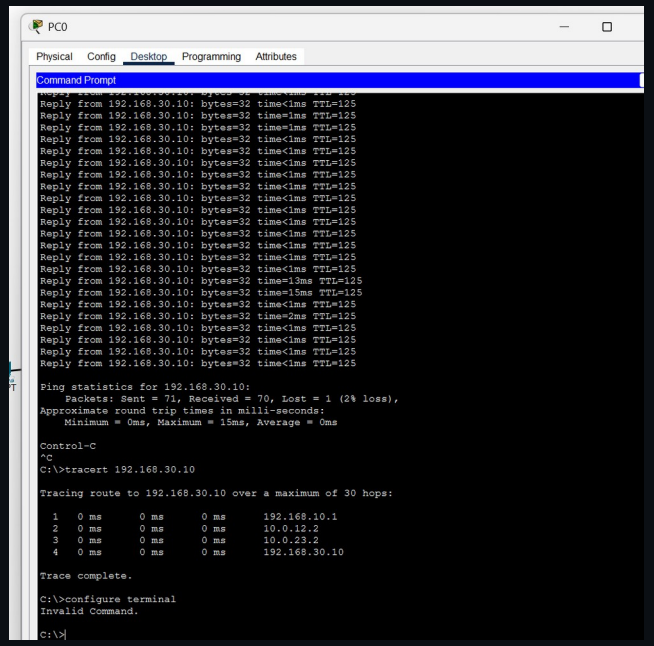
01:09:08: %OSPF-5-ADJCHG: Process 1, Nbr 2.2.2.2 on GigabitEthernet0/1 from LOADING to FULL,
Loading Done

01:09:08: %OSPF-5-ADJCHG: Process 1, Nbr 3.3.3.3 on GigabitEthernet0/2 from LOADING to FULL,
Loading Done

?Bad filename
%Error parsing filename (Bad file number)
R1#show ip route ospf
      10.0.0.0/8 is variably subnetted, 6 subnets, 2 masks
O        10.0.23.0 [110/2] via 10.0.12.2, 00:06:10, GigabitEthernet0/1
                   [110/2] via 10.0.13.2, 00:06:10, GigabitEthernet0/2
O        10.0.24.0 [110/2] via 10.0.12.2, 00:06:10, GigabitEthernet0/1
O     192.168.30.0 [110/2] via 10.0.13.2, 00:28:57, GigabitEthernet0/2
O     192.168.40.0 [110/3] via 10.0.12.2, 00:06:00, GigabitEthernet0/1

R1#
```

OSPF routes on R1



Link failure: 70/71 pings, reroute

```
Reply from 192.168.30.10: bytes=32 time=2ms TTL=125
Reply from 192.168.30.10: bytes=32 time<1ms TTL=125
Reply from 192.168.30.10: bytes=32 time<1ms TTL=125
Reply from 192.168.30.10: bytes=32 time<1ms TTL=125
Reply from 192.168.30.10: bytes=32 time<1ms TTL=125

Ping statistics for 192.168.30.10:
    Packets: Sent = 71, Received = 70, Lost = 1 (2% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 15ms, Average = 0ms

Control-C
^C
C:\>tracert 192.168.30.10

Tracing route to 192.168.30.10 over a maximum of 30 hops:

  1    0 ms      0 ms      0 ms      192.168.10.1
  2    0 ms      0 ms      0 ms      10.0.12.2
  3    0 ms      0 ms      0 ms      10.0.23.2
  4    0 ms      0 ms      0 ms      192.168.30.10

Trace complete.

C:\>configure terminal
Invalid Command.

C:\>ping 192.168.20.x
Ping request could not find host 192.168.20.x. Please check the name and try again.
C:\>ping 192.168.20.11

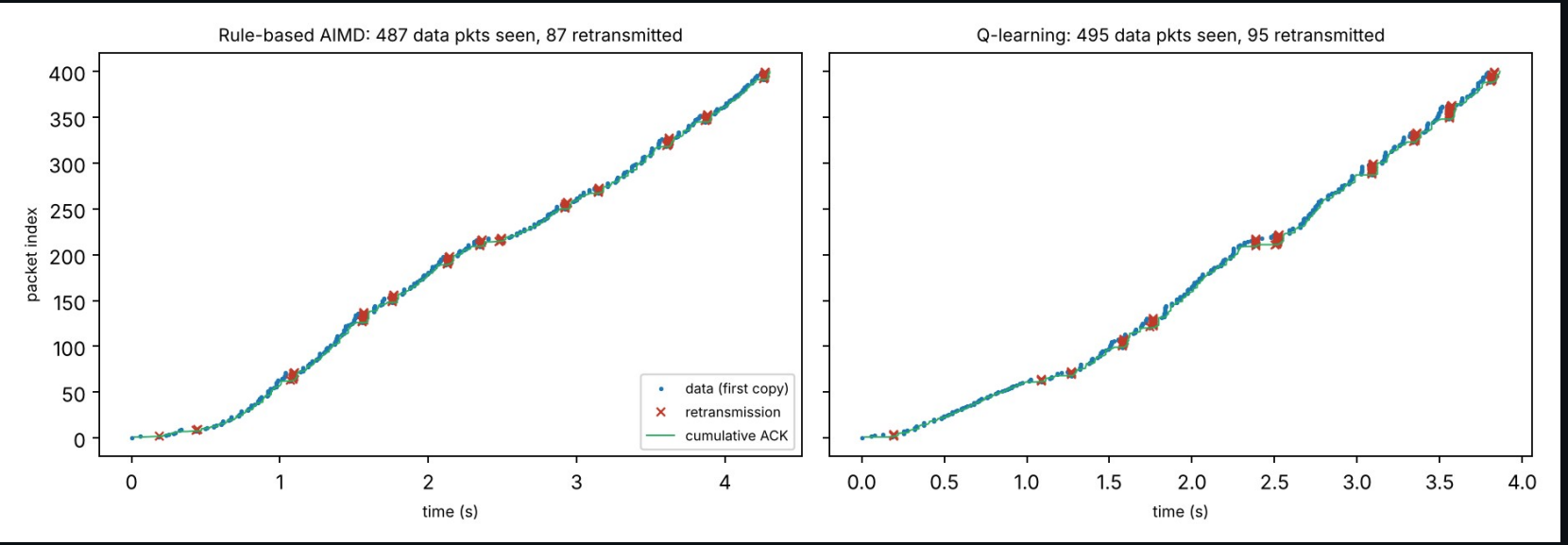
Pinging 192.168.20.11 with 32 bytes of data:

Reply from 192.168.10.1: Destination host unreachable.
Reply from 192.168.10.1: Destination host unreachable.
```

ACL: student blocked, staff allowed

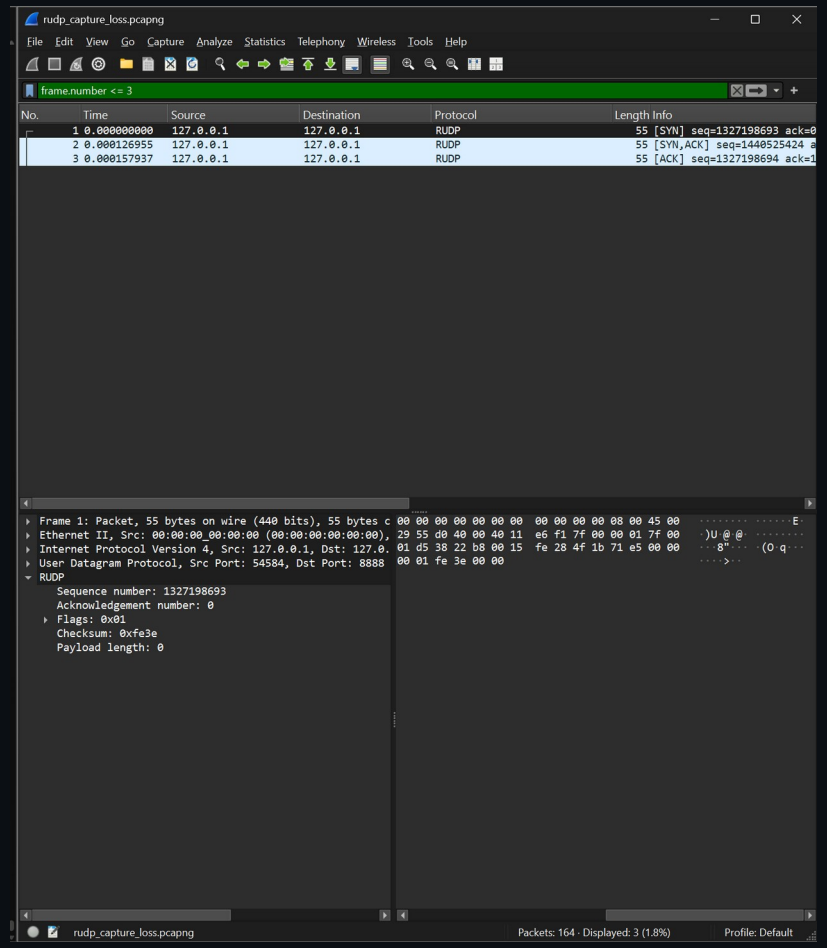
- Server LAN metric 2 via R3, branch LAN metric 3 via R2
- Shutting R1 Gi0/2: traffic reroutes via R2, only 1 of 71 pings lost
- Student PC gets "destination unreachable" to staff; server still reachable (TTL 126)

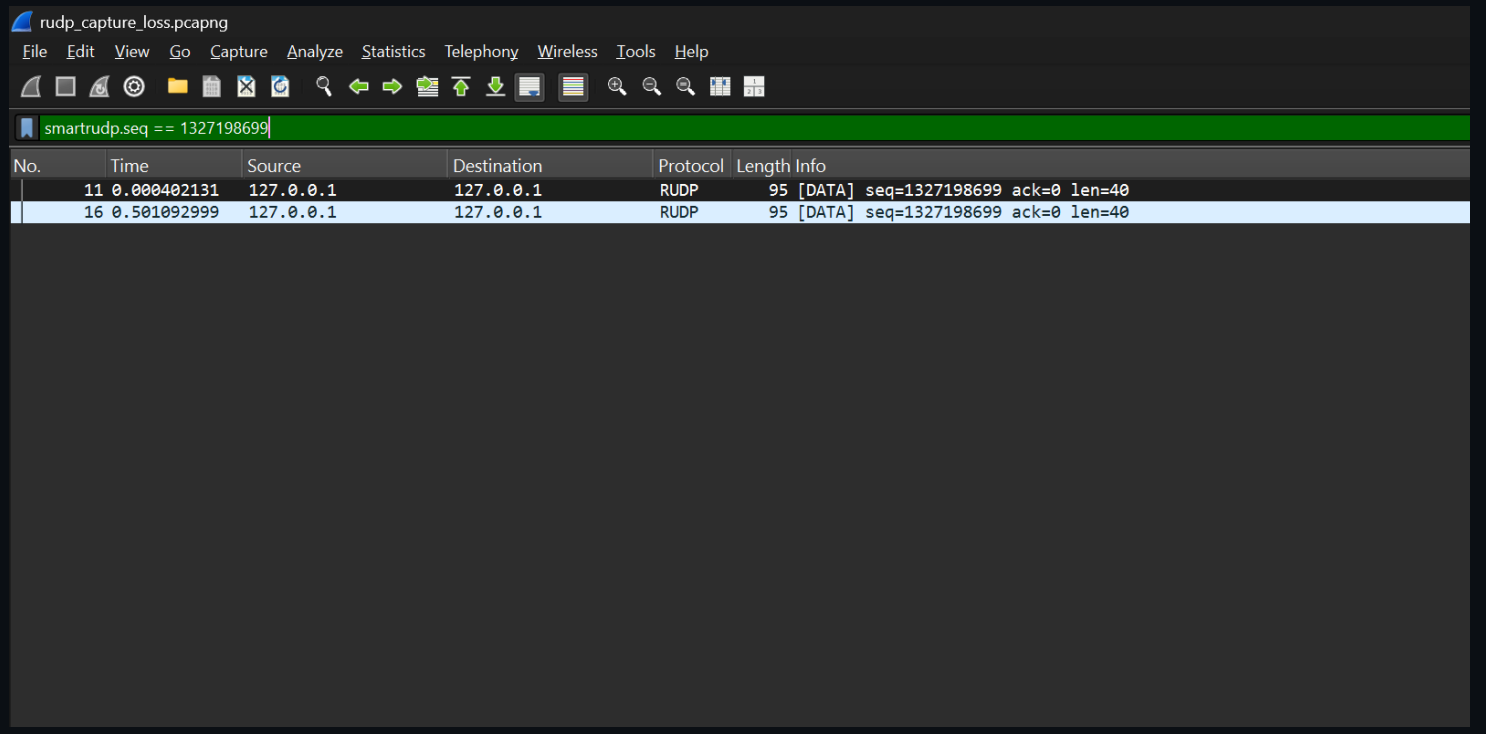
Wireshark: RUDP dissector and traces



- rudp.lua decodes seq, ack, flags, checksum, length
- Filters: smartrudp.flags.data == 1, smartrudp.seq == N
- 400 packets, lossy path: 87 (AIMD) and 95 (Smart) repeated data segments seen at the receiver
- Trace exported by the program, topology addresses, handshake not included

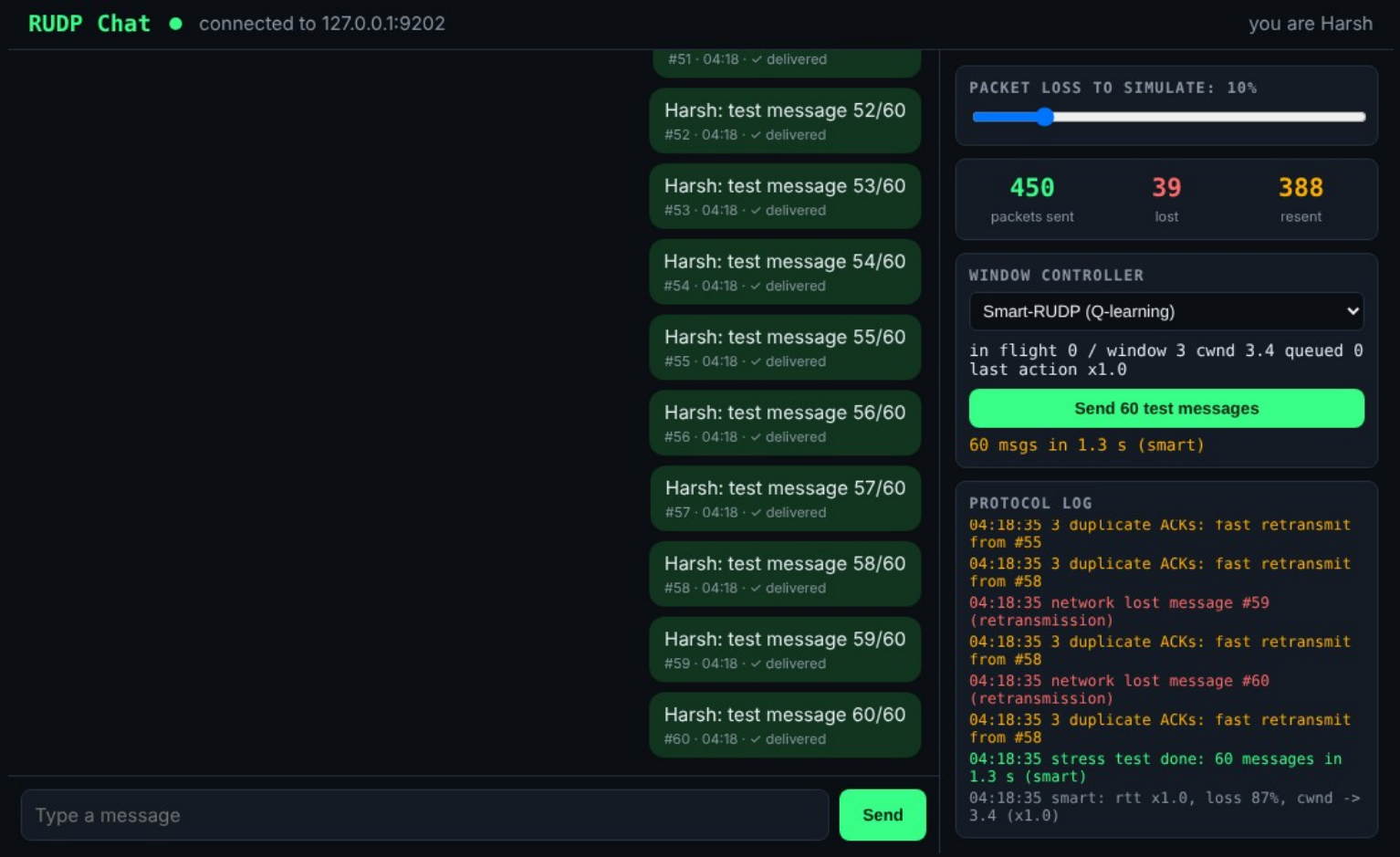
Wireshark: real capture





- Left: SYN, SYN-ACK, ACK decoded by the smartrudp dissector (filter frame.number <= 3)
- Right: smartrudp.seq == N shows the same segment sent twice, a retransmission

Demo: RUDP Chat over the internet



- Each message is one RUDP segment: numbered, checksummed, ACKed in order
- Browser UI via a small gateway; terminal version too
- Two machines linked over Tailscale (laptop and phone)
- Loss slider: lost messages are resent and still arrive in order
- Window controller switch: fixed, AIMD or Smart-RUDP (Q-learning, learns live)
- Retrained for the chat: faster than AIMD on 5 of 6 held-out paths, equal on 1 (Table 11)
- Stress test, 48 unseen paths: table 3.73 s, deep (DQN) 3.20 s, AIMD 7.16 s
- Deep agent is seed-sensitive, so picked on validation paths

The RUDP packet header: 13 bytes

Field	Size	Purpose
`seq`	4 B	Packet number
`ack`	4 B	Next packet expected
`flags`	1 B	SYN / ACK / FIN / DATA
`checksum`	2 B	Detects corruption
`payload_len`	2 B	Data length (max 512)

Fields packed byte by byte, so compiler struct padding never changes the wire format.

Data packet = 53 bytes on the wire. ACK / handshake = 13 bytes.

Handshake and data flow

1	**SYN**	Client	-> Server	Request a connection
2	**SYN-ACK**	Client	<- Server	Server agrees
3	**ACK**	Client	-> Server	ESTABLISHED, data may flow
4	**DATA**	Client	-> Server	Up to window-size packets in flight
5	**ACK**	Client	<- Server	Cumulative: confirms everything up to N
6	**FIN**	Client	-> Server	Close the connection

Server accepts a new SYN in any state, so a killed client can never leave it stuck.

Reliability: Go-Back-N

- Window = cwnd packets in flight (fixed at 4 in Phase 4, then driven by cwnd)
- Receiver sends cumulative ACKs
- Retransmission timeout: 500 ms
- Fast retransmit after 3 duplicate ACKs, no waiting for the timer
- Checksum failures are dropped, then recovered by retransmission

Proof

In a netem 20% loss capture we saw 12 timeouts spaced about 500 ms apart.

Result

2,442-byte message reassembled correctly.

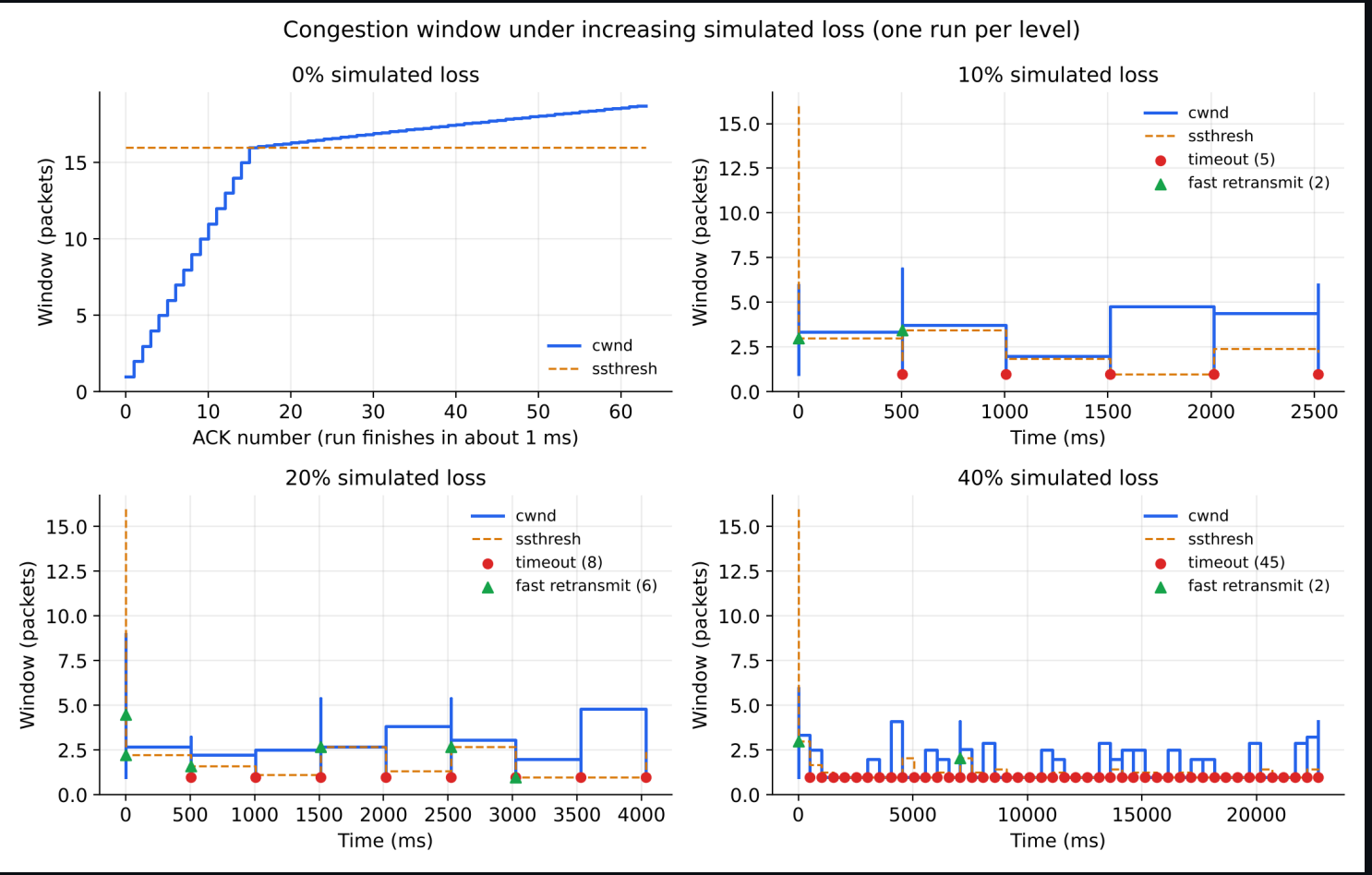
Reliability: Selective Repeat (the default)

C transport, seconds	AIMD 32	Neural	Gain
5G-like	1.27	1.16	9%
Lossy 2%	7.99	4.29	46%
Satellite-like	25.7	12.9	50%
Hostile, 10% loss	7.3	8.1	tie

- Receiver buffers out-of-order packets (64 window) and reports them in a bitmap in every ACK
- Sender: one timer per packet; resends only the missing ones
- Retransmissions drop about 10x; web chat 200 messages: 6.4 s vs 11.9 s (AIMD)
- The neural agent now wins by more, and the 10% loss failure was mostly a Go-Back-N effect
- Go-Back-N kept for comparison: --gbn / RUDP_GBN

10 runs per cell (hostile: 5), 95% intervals, all gains significant except the hostile row.
Code: eval_ci7.sh.

Congestion control: AIMD



- Slow start: cwnd +1 per ACK, below ssthresh 16
- Avoidance: cwnd + 1/cwnd per ACK
- 3 dup ACKs: cwnd halves
- Timeout: cwnd resets to 1, ssthresh halves
- Capped at 32

Fresh runs at 0%, 10%, 20% and 40% simulated loss. Red = timeout, green = fast retransmit.

Smart-RUDP: a Q-learning window controller

State (80)

RTT ratio x loss fraction x window size, sampled once per RTT.

Action (5)

Multiply cwnd by 0.5, 0.85, 1.0, 1.15 or 1.5.

Reward

Throughput gained, minus delay growth, minus retransmissions.

Same Go-Back-N reliability as the baseline; only the window policy changes. Paths emulated in-program: 5G-like, lossy wireless, satellite-like.

The AI agent in our code

src/rl_cc.h (agent, simplified)

```
// state: 4 x 4 x 5 = 80 buckets
s = (rtt_bucket*4 + loss_bucket)*5 + cwnd_bucket;

// choose: explore with prob. eps, else best
int rl_choose(a, s) {
  if (rand01() < a->eps) return rand() % 5;
  return argmax(a->q[s]);        // 5 actions
}

// learn: the Q-learning update
void rl_update(a, s, act, r, s2) {
  target = r + a->gamma * max(a->q[s2]);
  a->q[s][act] += a->alpha * (target - a->q[s][act]);
}
```

src/smart_client.c (decision loop, simplified)

```
// once per RTT, in smart_client.c
ratio = rtt / base_rtt;   lossf = losses / sent;

reward = sqrt(throughput_norm)
       - 0.5 * max(0, ratio - 1)   // delay growth
       - 4.0 * lossf;              // loss

rl_update(&ag, prev_s, prev_a, reward, s2);
act   = rl_choose(&ag, s2);
cwnd *= {0.5, 0.85, 1.0, 1.15, 1.5}[act];
```

Tabular Q-learning: alpha 0.15, gamma 0.9. Chat version: web/cc.py, trained in web/train.py (results/q_chat.txt). Deep version: web/dqn.py, a small neural network (5-32-32-5) trained by DQN in web/train_deep.py.

Neural agent in the C transport

1,200 packets, C transport	AIMD	Table (rl2)	Neural
5G-like	1.38 s	2.64 s	1.18 s
Lossy wireless	9.13 s	7.54 s	6.63 s
Satellite-like	17.9 s	38.1 s	13.1 s

15 runs per cell (satellite: 8), 95% intervals. Equal-cap check (AIMD capped at 32, 10 runs): neural still wins by 19%, 29%, 28%, all significant (eval_ci4.sh).

- Neural agent beats AIMD on all three paths: 15%, 27%, 24% faster (95% intervals exclude zero)
- Table agent does not generalise: slower on 5G-like and satellite-like
- Selective ACK in C: about 10x fewer retransmissions, no significant time change here
- Caveats: trained in a simulator; emulated paths (not real networks); agent retransmits more on random loss

Literature behind the design

- Jacobson & Karels (1988): congestion avoidance and control, the origin of AIMD
- Chiu & Jain (1989): why additive increase, multiplicative decrease converges fairly
- RFC 5681 (2009): TCP congestion control standard
- Ha, Rhee, Xu (2008): CUBIC, our future-work direction
- Langley et al. (2017) and RFC 9000: QUIC, reliable transport over UDP in production
- Midhula & Arun Raj Kumar (2024) and Cao et al. (2023, DRLFcc): deep reinforcement learning for congestion control, the idea behind Smart-RUDP

Key researcher: Van Jacobson

How it was tested

- Correctness: unit, end-to-end, handshake, file-integrity, chat and simulator tests (make test); packet parser fuzzed under AddressSanitizer and UBSan (make fuzz)
- Speed: virtual time, same C sender, checked against the real clock in 15 of 15 cells; paired seeds, 95% intervals, tuning seeds kept apart from test seeds
- Realism, in four steps: emulation in the client, a separate impairment proxy on real sockets, Linux kernel netem, and two laptops over the internet
- Every result in the report has the command that produced it (docs/RUNNING_ALL_TESTS.md)
- We also record our own mistakes: untuned baselines, unequal payloads and synthetic 40-byte packets were each caught, fixed and re-run

A fast, faithful C simulator, and what it showed



- --vt 1: the real C sender with a simulated receiver and clock, about 1000x faster; matches the real-clock runs in 15 of 15 cells
- 100 random held-out paths: neural agent 0.53x the time of tuned AIMD, faster on 89, never >20% slower (tuned CUBIC: slower by >20% on 27)
- Biggest factors: window cap 128, pacing, timer floor, slow start
- Evolution-strategy retraining in C: +2% only, not adopted

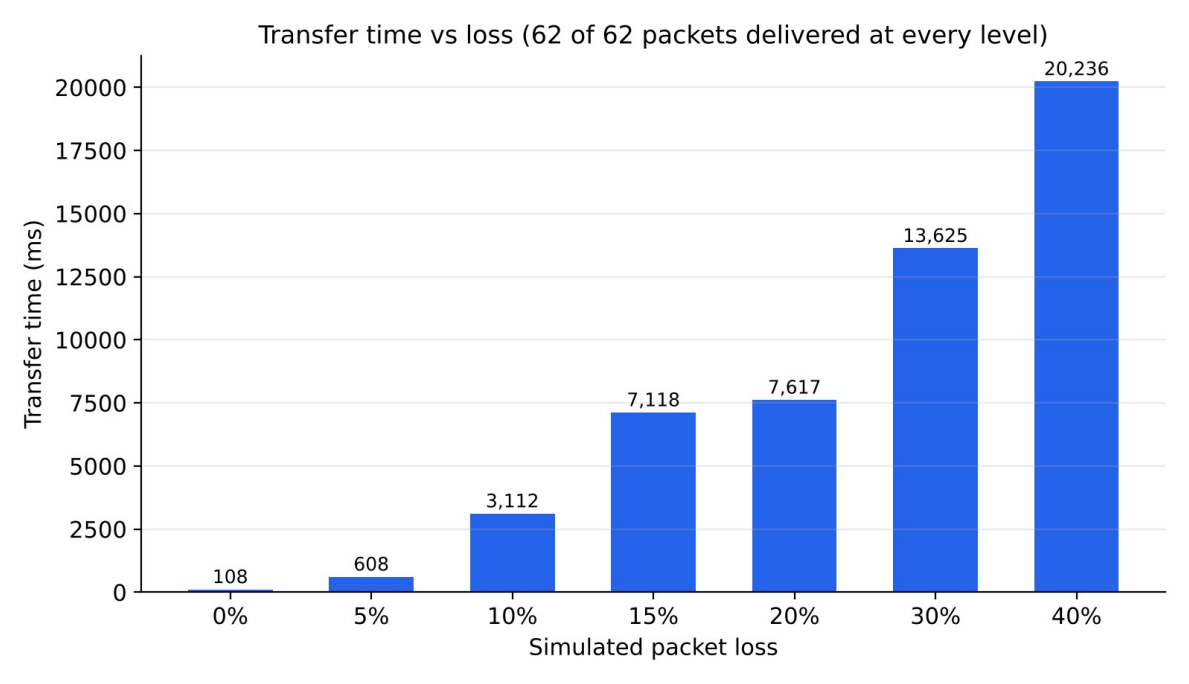
Issues we found along the way

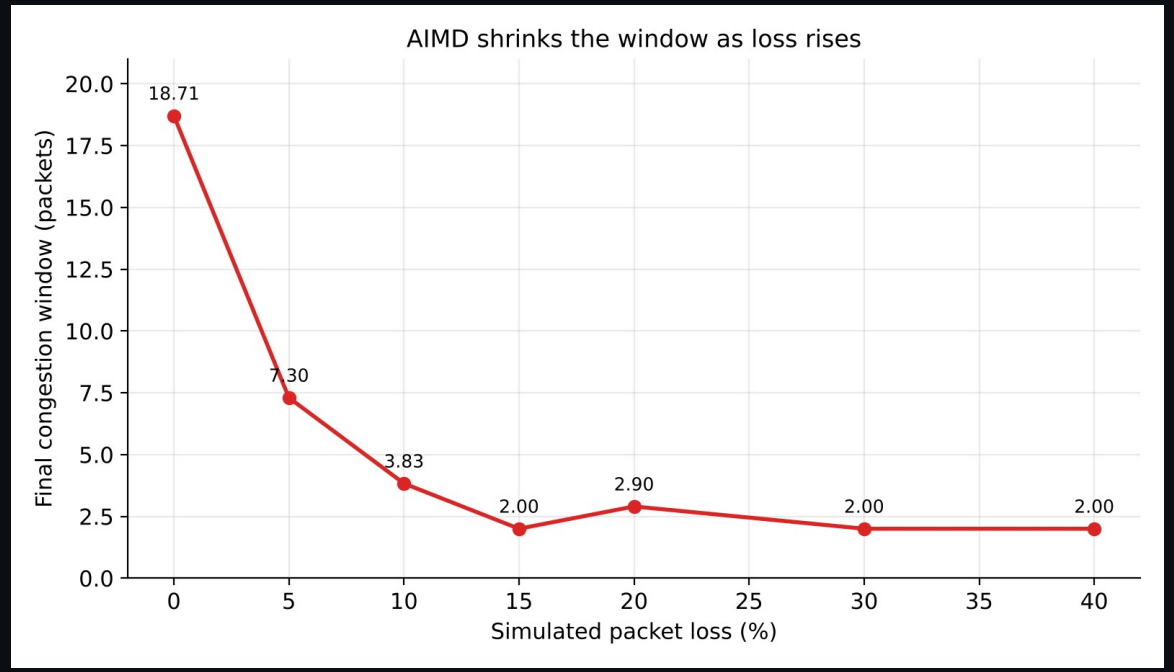
- The first learned controller (a Q-table) was not faster than AIMD
- At 10% random loss the neural agent was 7x slower than AIMD (85 s against 11 s)
- Real sockets exposed a lost-handshake hang, spurious timeouts (262 retransmissions for about 3 losses) and a weak checksum
- Our baselines were weak: untuned AIMD (window cap 32) made early wins look bigger than they are
- Our own kernel tests were wrong four times: netem limit, untuned baseline, unequal payloads, synthetic 40-byte packets
- The first BBR-style baseline stalled at 10% loss and took 163 s after a delay jump
- The agent is a pushier neighbour than AIMD or CUBIC, and results vary with the training seed

What we changed to make it better

Before	After
Go-Back-N with a fixed 500 ms timer	Selective Repeat: bitmap ACKs, per-packet timers, adaptive timer with a 1.3 x srtt floor; about 10x fewer retransmissions
Byte-sum checksum	CRC-16: 0 of 333k corrupted packets accepted
Server ignored data after a lost final handshake ACK	Handled as in TCP, with a regression test
Q-table agent, window cap 32	5-32-32-5 neural network, cap 128, slow start, pacing
Neural agent 7x slower than AIMD at 10% loss	Hand-over to AIMD while loss is high: 13.9 s instead of 84.9 s
Untuned AIMD and CUBIC baselines	Tuned baselines, so every win is against a fair opponent
Faulty kernel and BBR tests	Netem limit, equal payloads and real 512-byte packets; kernel runs repeated; BBR stall and slow recovery fixed

Benchmark results



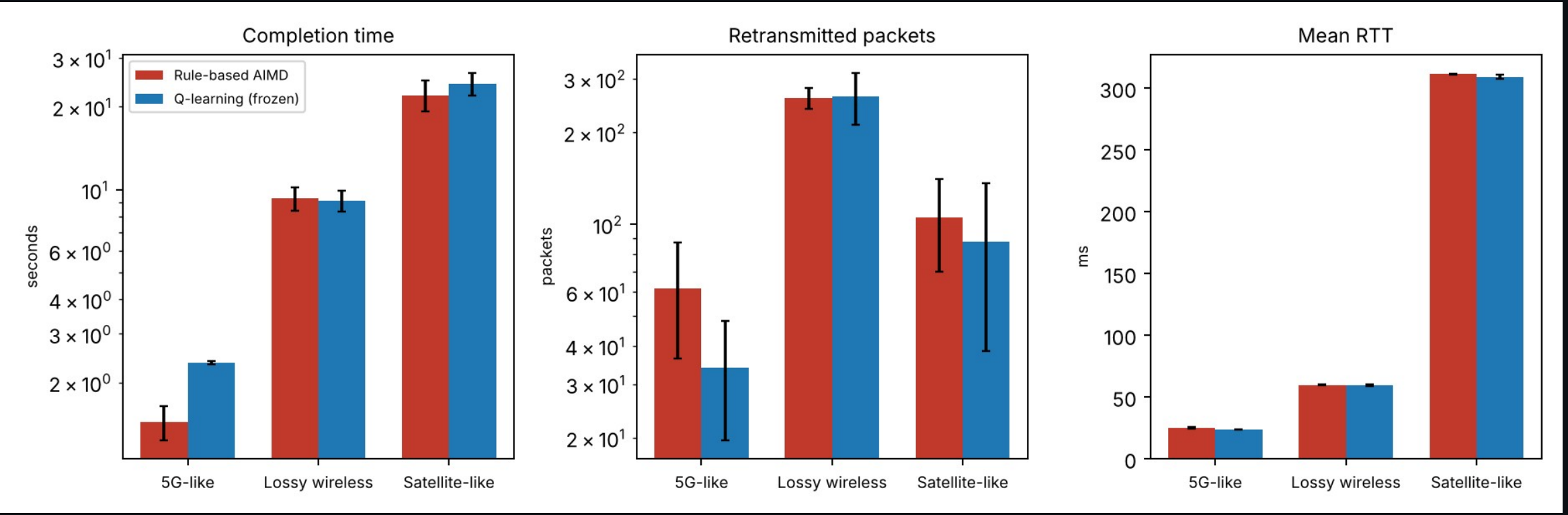


62 of 62 packets delivered at every loss level, from 0% to 40%.

Problems we hit and fixed

Problem	Fix
Server stuck after killed client	Accept SYN in any state (was a 15.5 s hang)
Leftover netem rule	Check tc qdisc show dev lo, delete before each test
Windows Wireshark saw nothing	Capture with tshark inside WSL, then copy
Shell portability in benchmark	Replaced date/grep tricks with awk and sed

Our first learned agent: an honest result



- 6 evaluation runs per scenario after 40 training episodes
- Not faster: +63% time on 5G-like, +10% on satellite, tie on lossy
- Fewer retransmissions on 5G-like and satellite, but within run-to-run noise
- Earlier training rounds were much worse; reported run is the final configuration

More evidence: baselines, SACK, reordering

48 paths, original simulator	Mean	Median
Plain UDP	68% delivered	-
TCP model (selective ACK)	4.56 s	1.48 s
RUDP fixed window	3.88 s	2.00 s
RUDP AIMD	7.16 s	2.72 s
Smart-RUDP, table	3.73 s	1.99 s
Smart-RUDP, deep	3.20 s	1.79 s

- Selective ACK (simulated): AIMD mean 7.16 s to 4.90 s, median 2.72 s to 1.44 s; faster on 34 of 48 paths. Go-Back-N is the main weakness.
- Reordering (20 ms jitter): AIMD 0.82 s to 6.79 s; deep agent 0.93 s; SACK removes most of the damage.
- 20-run intervals for the C experiments: Smart slower on 5G-like (+0.92 s) and satellite-like (+6.3 s), tie on lossy.

Caveat: this simulator allowed a dup-ACK retransmission storm. With the realistic rule, stress means are AIMD 7.75 s, neural 7.27 s: about level. The TCP row is a model, not kernel TCP.

Real sockets found what the emulator could not

Lost handshake ACK

Server ignored all data after a lost final ACK: transfers hung (2 of 4 runs at 10% loss). Fixed as in TCP; regression test added.

Spurious timeouts

Timer too tight for window bursts: 262 retransmissions for ~3 losses. Floor of 1.3 x srtt. It also made AIMD 41% faster, so the old margins were too big.

Weak checksum

Byte-sum checksum let 2% of corrupted packets through (fuzzer). Now CRC-16: 0 of 333k accepted, sanitizers clean.

Independent check on real UDP sockets through an impairment proxy (src/impair.c): neural agent -17% / -53% / -45% / -39% vs tuned AIMD on 5G / lossy / satellite / fast; hostile path a tie. eval_proxy.sh

Safety net: hand over to AIMD

C transport, seconds	AIMD 32	Neural	Hybrid
5G-like	1.48	1.17	1.23
Lossy 2%	9.04	7.53	7.57
Satellite-like	19.7	13.4	13.7
Hostile, 10% loss	11.1	84.9	13.9

10 runs per cell (hostile: 5 runs, 300 packets), 95% intervals. Flag: --hybrid 0.10. Code: eval_ci6.sh.

- Problem found: at 10% loss the neural agent is 7x slower than AIMD
- Fix: while a decision interval loses 10% or more, AIMD drives the window; after 3 clean intervals the agent resumes
- Keeps the wins on the three benign paths, and the hostile path is no longer significantly different from AIMD
- A timeout cut (x0.25) was tried and dropped: it helped the hostile path but hurt the 2% path (24 s vs 9 s)
- Honest summary: the agent helps on clean-to-moderate paths; it needs a classical safety net elsewhere

Fairness, ablations, chat fixes

Two flows, one bottleneck	Share of A	Jain
AIMD vs AIMD (long path)	51%	0.999
Deep vs deep (long path)	59%	0.969
Deep vs AIMD (no-loss path)	44%	0.987
Deep vs AIMD (long path)	74%	0.819
Fixed 8 vs AIMD (long path)	72%	0.843

Simulator, 20 seeds; ablation on 48 stress paths. Code: web/fairness_test.py, web/ablate.py.

- Fair on short paths; takes 74% against AIMD on the 120 ms path, like a fixed window
- Ablation: the no-shrink mask is what matters (6.54 s without it, about 3.3 s with it; AIMD 7.16 s)
- Other parts (delay penalty, last action, throughput input) showed no measurable value
- Same recipe gave 3.2 to 7.4 s across seeds: seed sensitivity is a real weakness
- Found and fixed a dup-ACK retransmission storm in the chat gateway; added a live dashboard and an adaptive RTO

Fairness and file transfer

Two flows, real sockets	Share A	Share B	Jain
AIMD vs AIMD	49%	51%	0.99
Neural vs neural	50%	50%	0.97
Neural vs AIMD tuned	68%	32%	0.88
Neural vs AIMD cap 32	76%	24%	0.78
CUBIC vs AIMD tuned	61%	39%	0.93

File transfer	AIMD cap 32	AIMD tuned	Neural
2 MB, lossy 2%	26.6 s	24.2 s	11.9 s
4 MB, fast path	34.6 s	21.0 s	8.9 s

- The agent is a more aggressive neighbour than AIMD or CUBIC: no starvation, but not polite. Not trained for fairness; TCP coexistence untested
- smart_client --file sends a real file; the received file is byte-identical in every run (FNV-1a hash)
- Terminal chat and web gateway now use the same 256-packet window and 32-byte bitmap as the C transport; interop test at 20% loss
- CI, fuzzing, MIT licence, architecture doc

Closing gaps: fairness and cross traffic

Neural vs tuned AIMD	Agent share	Jain
Default	70%	0.86
--polite 1.2	49%	0.95
--polite 1.3	52%	0.94
--polite 1.5	59%	0.90

Bursty cross traffic	Time	vs AIMD
AIMD tuned	5.73 s	1.00
CUBIC tuned	4.55 s	0.79
Neural agent	4.87 s	0.85
Neural, polite 1.3	4.92 s	0.86

- Polite mode: back off when a standing queue builds (jitter-aware). Share 70% to 52%; costs speed (0.53x to 0.58x of AIMD time on 100 paths, worst path 2.95x slower), so it is off by default
- Bursty cross traffic (new proxy option, 8 runs): neural is 15% faster than tuned AIMD but level with CUBIC; three tweaks to fix its extra retransmissions did not help
- Server now handles up to 16 clients at once; a test sends two files simultaneously and checks both byte-for-byte

Limits we state openly

- Kernel TCP is still 1.5x to 1.7x faster than the neural agent on every kernel path
- Against tuned AIMD the agent takes about 70% of a shared link; the optional polite mode fixes that but costs speed
- Level with CUBIC under bursty cross traffic; no gain at 10% random loss; the BBR-style baseline is faster on the 2% lossy path
- Not formally verified: certified at +-0.001 for only 43% of decisions; trained in a simulator, never on real traces
- Two-machine run: one drifting relay path, 4 of 20 runs timed out, so no ranking is claimed
- Coexistence with kernel TCP: 3 runs per cell

Learnings and future work

Learned

Reliability is a set of simple rules working together.
Congestion control matters as much as retransmission.
Testing tools fail in surprising ways.

Next

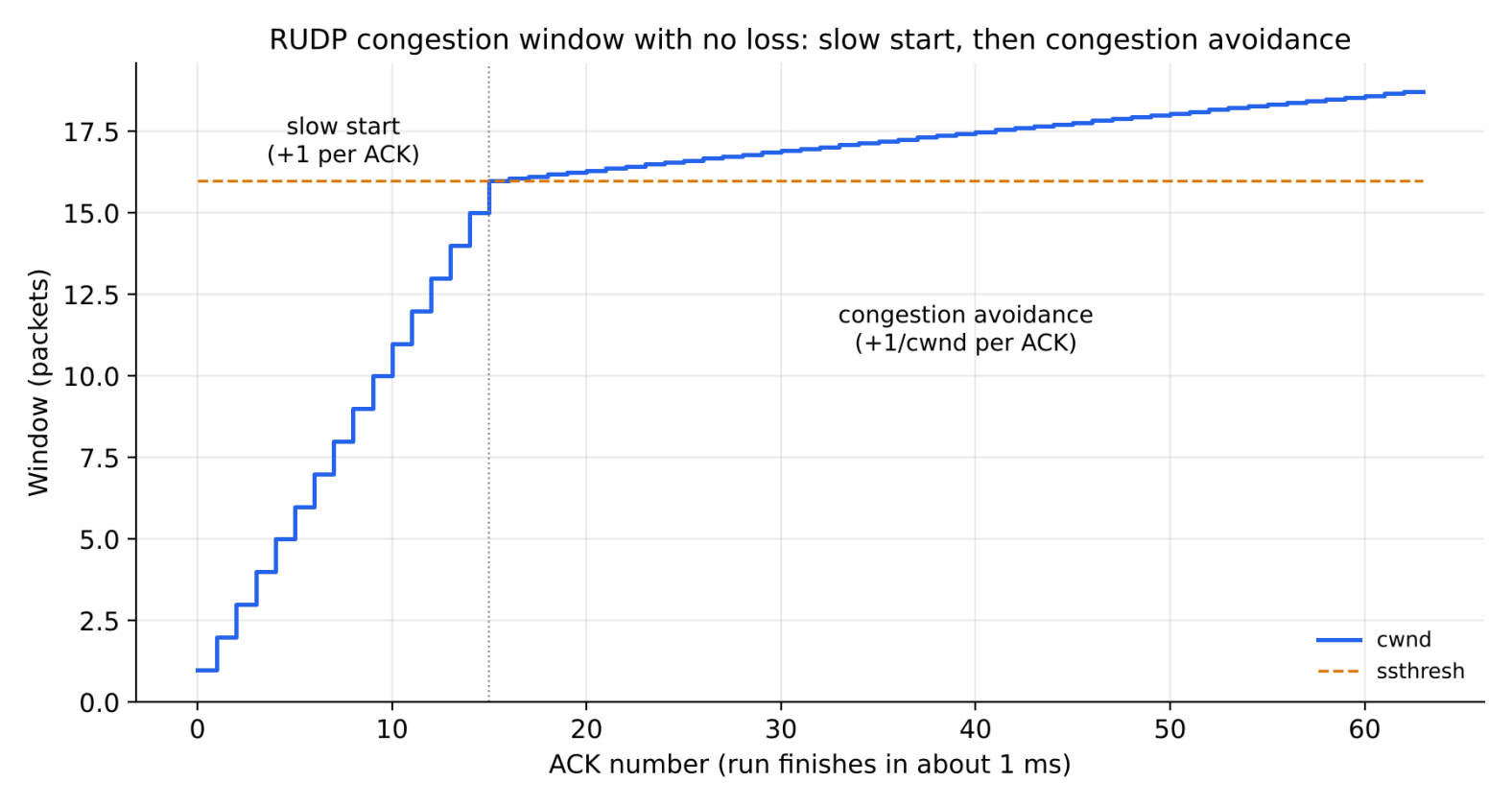
Fairness-aware training, TCP coexistence
Train on real traces with cross traffic
Close the gap to kernel TCP; controlled multi-machine tests

Thank you

`github.com/Capsturrrr/rudp`

Harshavardhan Mallela Rohit Arya Shloak Sinha

Appendix: slow start and congestion avoidance



- cwnd starts at 1
- +1 per ACK until ssthresh = 16
- Then +1/cwnd per ACK
- 62 packets, final cwnd 18.71